

Aegis VANGUARD

Système de Détection et de Correction de Vulnérabilités par IA Hybride

---

Projet de Master

Sécurité Informatique & Intelligence Artificielle

MERBOUH Aymane

BOUJIDA Yassine

IBN EL QORCHY Aymane

INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

Année universitaire 2025–2026

Table des matières

1 Introduction et Contexte **3**
- 1.1 La cybersécurité : un enjeu critique . . . 3
- 1.2 Le problème : l'explosion des vulnérabilités . . . 3
- 1.3 Objectif du projet . . . 4

2 État de l'Art **5**
- 2.1 Les approches traditionnelles (SAST) . . . 5
- 2.2 L'apport du Deep Learning . . . 5
  - 2.2.1 Historique : CNN et LSTM . . . 5
  - 2.2.2 La révolution des Transformers : CodeBERT . . . 6
- 2.3 Les concepts avancés (Fondements théoriques) . . . 7
  - 2.3.1 PLBART : Encoder-Decoder pour le code . . . 7
  - 2.3.2 RAG : Retrieval-Augmented Generation . . . 7
  - 2.3.3 LLM récents comme outils de validation . . . 8

3 Gestion de Projet : Méthodologie Agile **9**
- 3.1 Organisation de l'équipe . . . 9
- 3.2 Approche itérative : Sprints . . . 9
- 3.3 Outils de gestion . . . 10

4 Données et Choix Technologiques **11**
- 4.1 Comparatif des datasets . . . 11
- 4.2 Le choix final : DiverseVul . . . 11
- 4.3 Stack technique . . . 14

5 Architecture Globale du Système **15**
- 5.1 Couche 1 : Data Layer (Préparation) . . . 15
  - 5.1.1 Extraction des fonctions . . . 15
  - 5.1.2 Nettoyage et normalisation . . . 15
  - 5.1.3 Tokenization via CodeBERT Tokenizer . . . 16
- 5.2 Couche 2 : AI Model Layer (Détection) . . . 16
  - 5.2.1 Fine-tuning de CodeBERT . . . 16
  - 5.2.2 Sortie : Probabilité de vulnérabilité . . . 16
  - 5.2.3 Decision Module : Système de seuil . . . 17
- 5.3 Couche 3 : Application Layer (Vérification & Interaction) . . . 17
  - 5.3.1 Innovation majeure : LLM Verification Layer . . . 17
  - 5.3.2 Génération de correctifs . . . 18
  - 5.3.3 Backend API . . . 18
  - 5.3.4 Frontend : Extension VS Code et Interface Web . . . 18

6 Réalisation et Implémentation **20**
6.1 Entraînement de CodeBERT . . . 20
6.1.1 Paramètres d'entraînement . . . 20
6.1.2 Focal Loss . . . 20
6.2 Le pipeline de vérification . . . 21
6.2.1 Prompt engineering . . . 21
6.2.2 Pipeline de remédiation à deux niveaux . . . 21
6.3 L'extension VS Code . . . 22
6.3.1 Architecture de l'extension . . . 22
6.3.2 Communication avec l'API . . . 22

7 Évaluation et Résultats Expérimentaux **24**
7.1 Courbes d'apprentissage . . . 24
7.2 Résultats des différents entraînements . . . 25
7.2.1 Premier entraînement exploratoire (Aymane M.) . . . 26
7.2.2 Entraînement avec Focal Loss (Aymane M.) . . . 27
7.2.3 Entraînement pondéré sur Colab (Yassine B.) . . . 27
7.3 Analyse des métriques . . . 28
7.4 Analyse critique des résultats . . . 29
7.4.1 Pourquoi le rappel est excellent . . . 29
7.4.2 Pourquoi la précision est faible . . . 29
7.4.3 Stratégie de défense : mieux vaut trop d'alertes . . . 30
7.4.4 Le rôle du LLM : le filtre expert . . . 30

8 Conclusion et Perspectives **31**
8.1 Bilan . . . 31
8.2 Limitations . . . 31
8.3 Perspectives . . . 32

Chapitre 1

Introduction et Contexte

1.1 La cybersécurité : un enjeu critique

Chiffre clé

« Une cyberattaque a lieu toutes les 39 secondes dans le monde. »
— Université du Maryland / IBM Security

La cybersécurité est devenue un pilier incontournable du développement logiciel moderne. Les attaques informatiques se multiplient à un rythme sans précédent, ciblant aussi bien les grandes entreprises que les infrastructures critiques et les PME. Les conséquences sont considérables : pertes financières, atteintes à la vie privée, interruption de services essentiels.

1.2 Le problème : l'explosion des vulnérabilités

Le nombre de CVE (*Common Vulnerabilities and Exposures*) enregistrées chaque année ne cesse de croître, dépassant les **25 000 nouvelles vulnérabilités par an**. Face à cette explosion, les équipes de développement sont dans l'incapacité de vérifier manuellement chaque ligne de code.

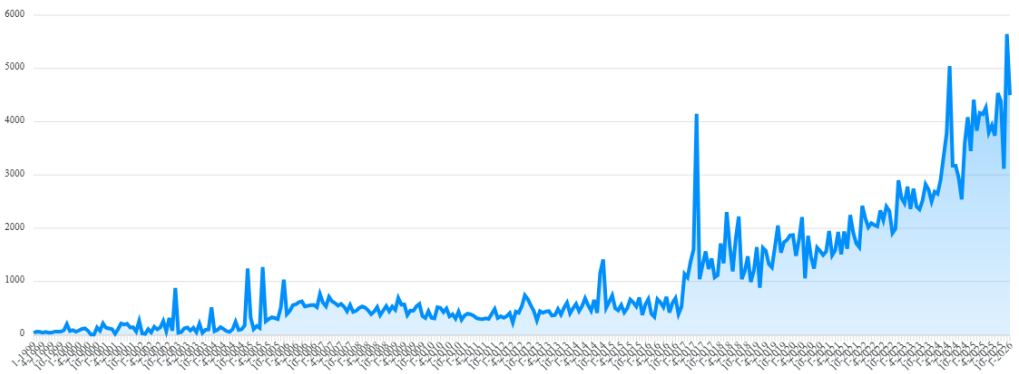


FIGURE 1.1 – Évolution du nombre de CVE publiées par année

Vulnérabilités critiques ciblées

Le projet se concentre sur les failles les plus dangereuses et les plus répandues :

- **Buffer Overflow** — Dépassement de mémoire tampon (heap et stack)
- **SQL Injection** — Injection de requêtes SQL malveillantes
- **XSS** (*Cross-Site Scripting*) — Injection de scripts dans les pages web
- **Null Pointer Dereference** — Accès à un pointeur non initialisé
- **Use-After-Free** — Utilisation de mémoire après libération

1.3 Objectif du projet

L'objectif de ce projet de Master est de concevoir et développer **Aegis VANGUARD**, un système hybride intelligent capable de :

1. **Détecter** automatiquement les vulnérabilités dans le code source grâce à un modèle CodeBERT fine-tuné.
2. **Vérifier** les alertes via une couche de validation par LLM pour réduire les faux positifs.
3. **Corriger** le code vulnérable en proposant des patchs générés par IA.
4. **Intégrer** l'ensemble dans un environnement de développement (VS Code) et une interface web.

Chapitre 2

État de l'Art

2.1 Les approches traditionnelles (SAST)

Les outils d'analyse statique de sécurité (*Static Application Security Testing*) constituent la première ligne de défense pour la détection de vulnérabilités.

TABLE 2.1 – Comparatif des outils SAST traditionnels

Outil	Forces	Limites
SonarQube	Open-source, intégration CI/CD facile	Règles rigides, faux positifs élevés
Checkmarx	Couverture multi-langages, très complet	Propriétaire, coûteux, lent sur de gros projets
Fortify	Standard industriel, certifié	Résultats difficiles à trier, configuration complexe

Limites fondamentales des approches traditionnelles

— **Taux élevé de faux positifs** — Les équipes perdent du temps à trier les alertes non pertinentes.

— **Pas de compréhension contextuelle** — Les règles syntaxiques ne capturent pas la sémantique du code.

— **Maintenance lourde** — Chaque nouvelle classe de vulnérabilité nécessite la rédaction manuelle de règles.

2.2 L'apport du Deep Learning

2.2.1 Historique : CNN et LSTM

Les premières approches de détection de vulnérabilités par deep learning ont exploité les réseaux de neurones convolutifs (**CNN**) et les réseaux récurrents à mémoire longue (**LSTM**). Bien que prometteurs, ces modèles souffrent de limitations dans la capture des dépendances longue portée dans le code.

2.2.2 La révolution des Transformers : CodeBERT

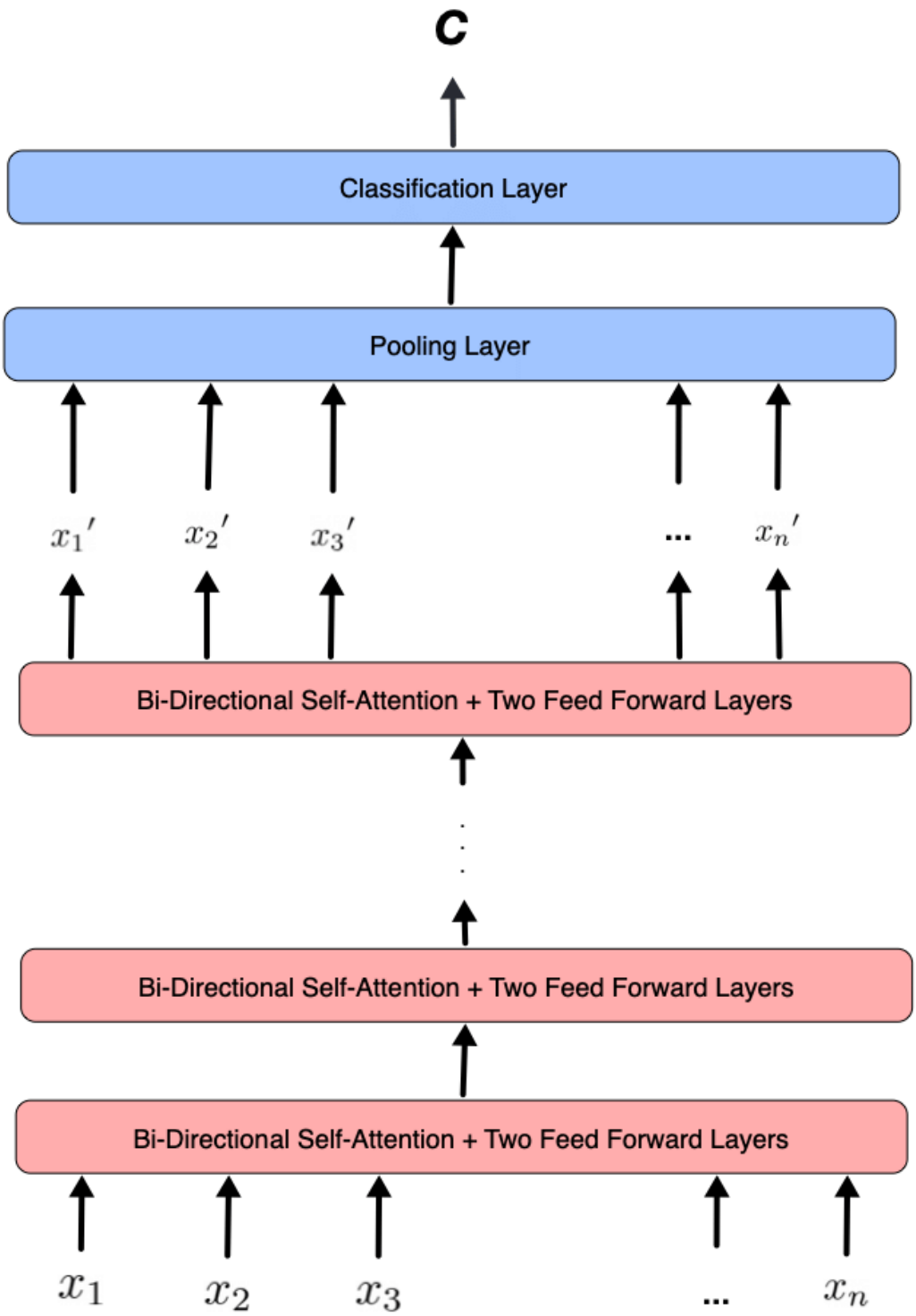


FIGURE 2.1 – Architecture de CodeBERT — Modèle Encoder-only bidirectionnel

CodeBERT (Microsoft Research, 2020) est un modèle pré-entraîné basé sur l'architecture **RoBERTa** (variante optimisée de BERT). Il introduit une compréhension **bidirectionnelle** du code : chaque token est analysé en fonction de tout son contexte (gauche

et droite), contrairement aux modèles auto-régressifs.

Pourquoi CodeBERT ?

- **Encoder-only** — Optimisé pour la classification (pas la génération)
- **Bidirectionnel** — Comprend le contexte complet du code
- **Pré-entraîné sur 6 langages** — Python, Java, JavaScript, PHP, Ruby, Go
- **125 millions de paramètres** — Assez puissant, mais déployable sur des machines modestes

2.3 Les concepts avancés (Fondements théoriques)

2.3.1 PLBART : Encoder-Decoder pour le code

PLBART (*Program and Language BART*) étend l'approche au paradigme **Encoder-Decoder**, permettant non seulement de comprendre le code mais aussi d'en **générer**. Cette architecture est pertinente pour la génération de patchs correctifs.

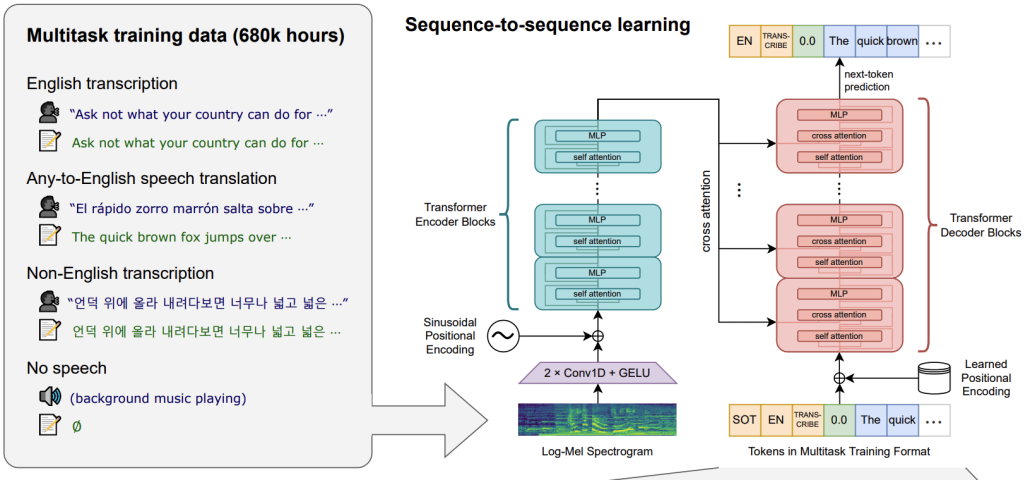


FIGURE 2.2 – Architecture Encoder-Decoder de PLBART

2.3.2 RAG : Retrieval-Augmented Generation

Le **RAG** (*Retrieval-Augmented Generation*) est une technique qui enrichit les réponses d'un LLM en lui fournissant des informations issues de **bases de connaissances externes**. Dans le contexte de la cybersécurité, cela permet de :

- Consulter une base de CVE connues avant de valider une alerte
- Enrichir les explications avec des références précises (CWE, OWASP)
- Proposer des corrections basées sur des bonnes pratiques documentées

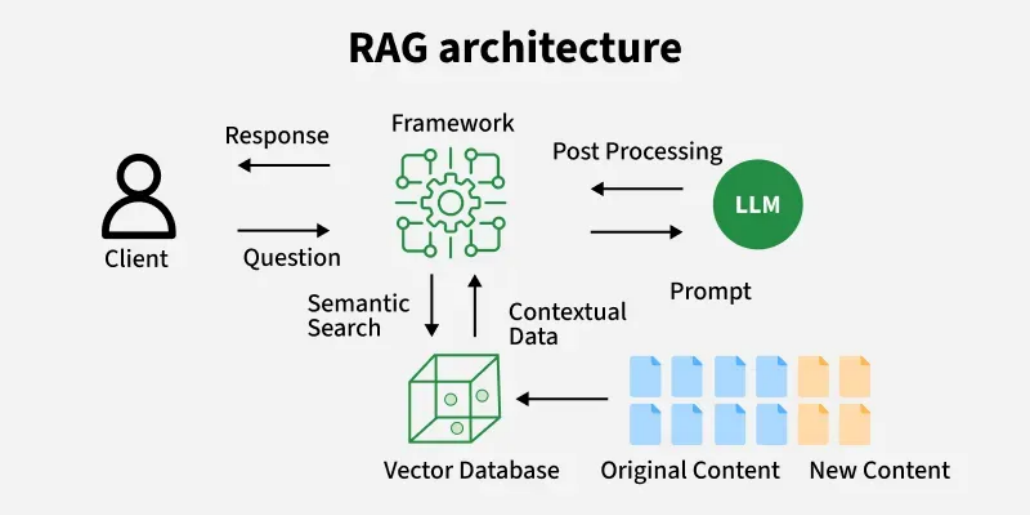


FIGURE 2.3 – Principe du RAG appliqué à la détection de vulnérabilités

2.3.3 LLM récents comme outils de validation

Les grands modèles de langage récents (**GPT-4o**, **Claude Opus**, **Gemini**) ne sont pas utilisés comme modèle principal de détection, mais comme **couche de vérification**. Leur capacité de raisonnement sur le code permet de :

1. Confirmer ou infirmer une alerte de CodeBERT (réduction des faux positifs)
2. Expliquer *pourquoi* le code est vulnérable en langage naturel
3. Proposer un correctif contextualisé

Chapitre 3

Gestion de Projet : Méthodologie Agile

3.1 Organisation de l'équipe

TABLE 3.1 – Répartition des rôles au sein de l'équipe

Membre	Responsabilités
MERBOUH Aymane	Architecture IA, entraînement CodeBERT, analyse des résultats
BOUJIDA Yassine	Développement de l'extension VS Code, API backend, déploiement
IBN EL QORCHY Aymane	Étude des datasets, prétraitement des données, documentation

3.2 Approche itérative : Sprints

Le projet a suivi une méthodologie **Agile** organisée en 4 sprints :

Sprint 1 — Étude et Choix Technologiques

- Étude comparative des datasets (DiverseVul, Devign, Defectors, PrimeVul)
- Analyse de l'état de l'art (CodeBERT, PLBART, RAG)
- Choix de la stack technique (PyTorch, Transformers, Flask)

Sprint 2 — Pipeline CodeBERT

- Prétraitement du dataset DiverseVul
- Fine-tuning de CodeBERT pour la classification binaire
- Évaluation des métriques (Accuracy, Recall, Precision, AUC)

Sprint 3 — Architecture Hybride

- Intégration de la couche de vérification LLM
- Conception du pipeline de validation (CodeBERT → LLM)
- Prompt engineering pour la remédiation

Sprint 4 — Produit Final

- Développement de l'extension VS Code
- Développement de l'interface web
- Déploiement de l'API sur Hugging Face Spaces
- Rédaction du rapport et préparation de la soutenance

3.3 Outils de gestion

TABLE 3.2 – Outils utilisés pour la gestion du projet

Catégorie	Outil
Gestion de projet	Jira / Trello
Versioning	GitHub
Calcul	Google Colab / GCP
Communication	Discord / Google Meet
Documentation	LaTeX / Overleaf

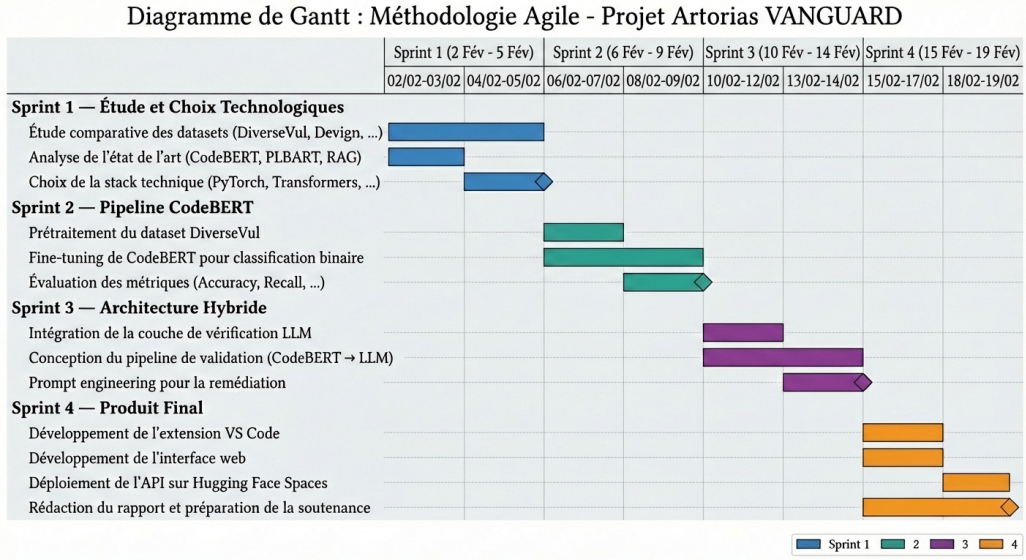


FIGURE 3.1 – Diagramme de Gantt — Planification des sprints

Chapitre 4

Données et Choix Technologiques

4.1 Comparatif des datasets

Plusieurs datasets de vulnérabilités ont été évalués avant de faire un choix :

TABLE 4.1 – Comparatif des datasets de vulnérabilités logicielles

Dataset	Taille	Langages	Caractéristiques	Retenu ?
Defectors	218k	Python	Généraliste, faible diversité de vulnérabilités	✗
Devign	27k	C	Trop petit pour fine-tuner un Transformer	✗
PrimeVul	236k	C/C++	Bon, mais moins diversifié que DiverseVul	✗
DiverseVul	**350k**	**C/C++**	**Grande diversité, vrais CVE, well-maintained**	✓

4.2 Le choix final : DiverseVul

Pourquoi DiverseVul ?

1. **Grande taille** — 350 000 fonctions C/C++ étiquetées
2. **Diversité** — Couvre un large spectre de types de vulnérabilités (CWE)
3. **Source fiable** — Basé sur de vrais CVE publics issus du NVD
4. **Spécificité C/C++** — Cible les langages système les plus exposés aux failles mémoire
5. **Qualité** — Nettoyage et étiquetage rigoureux par les auteurs

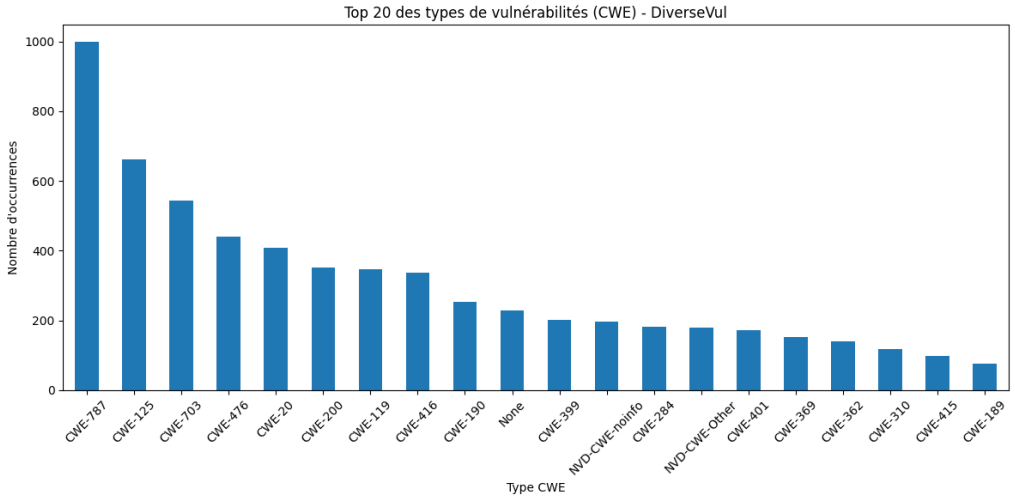


FIGURE 4.1 – Distribution des types de vulnérabilités dans DiverseVul

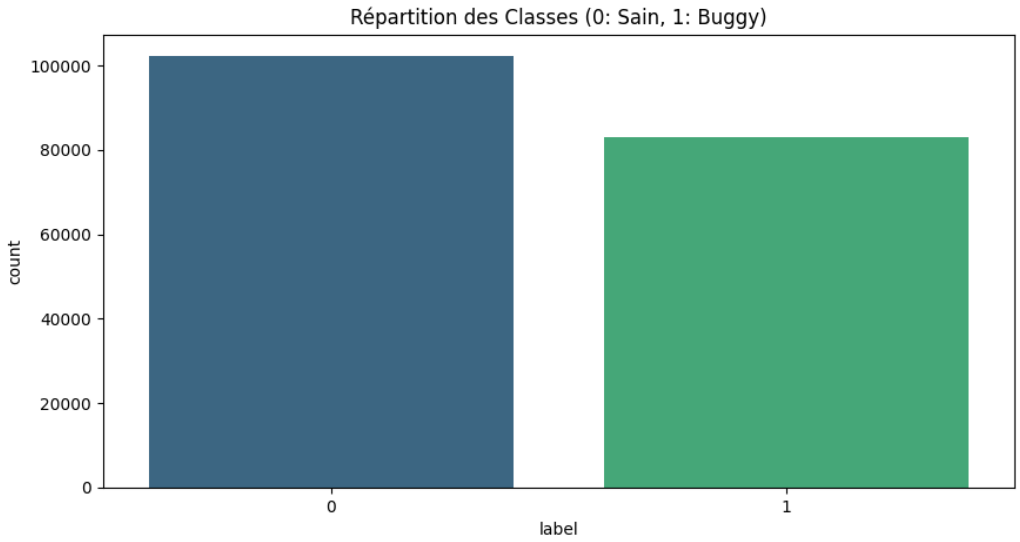


FIGURE 4.2 – Équilibre des classes dans DiverseVul — Répartition sain vs buggé

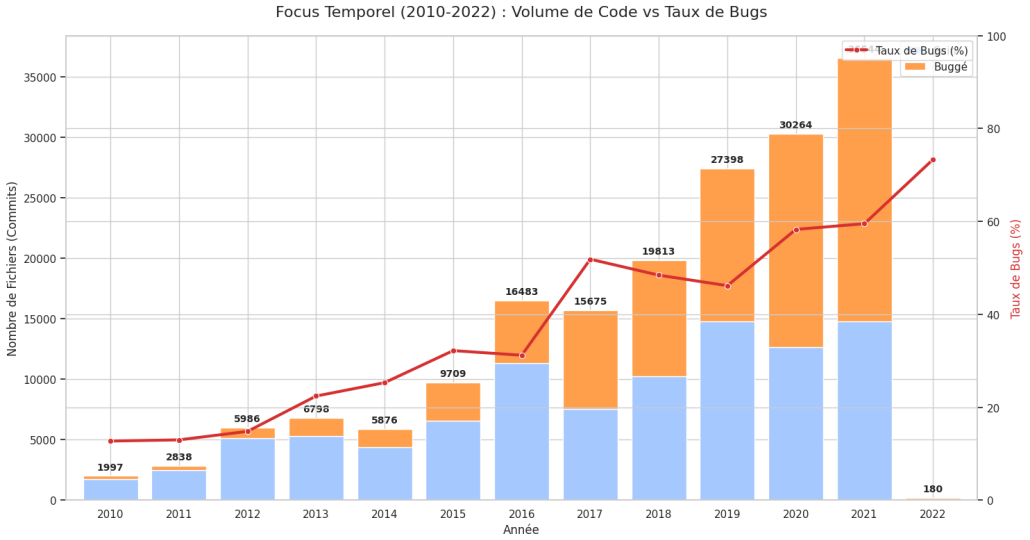


FIGURE 4.3 – Focus temporel (2010–2022) — Volume de code vs taux de bugs

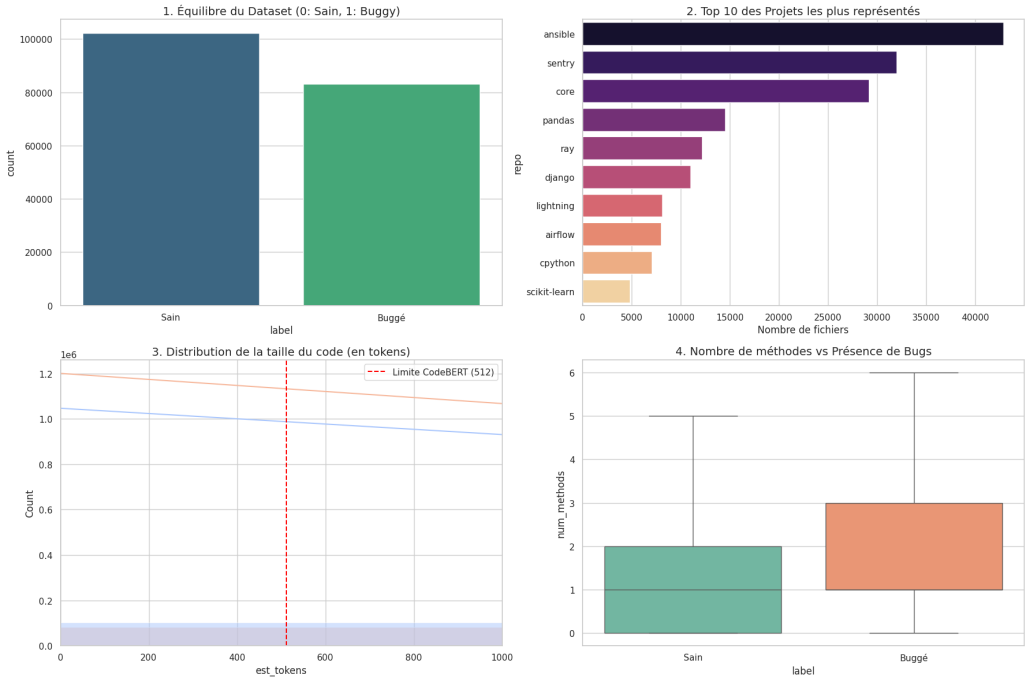


FIGURE 4.4 – Exploration du dataset DiverseVul — Top projets, distribution des tokens, nombre de méthodes

4.3 Stack technique

TABLE 4.2 – Stack technique du projet Aegis VANGUARD

Catégorie	**Technologie**	**Justification**
Langage	Python 3.10+	Écosystème ML/DL mature
Framework IA	PyTorch	Flexibilité, communauté active
Modèles	Hugging Face Transformers	Accès direct à CodeBERT
Modèle core	CodeBERT (Microsoft)	Optimisé pour la compréhension du code
Backend	Flask	Léger et suffisant pour servir le modèle
Frontend	HTML/CSS/JS	Interface web dark theme
Extension IDE	TypeScript / VS Code API	Intégration native dans l'éditeur
Déploiement	Hugging Face Spaces (Docker)	Gratuit, GPU disponible
Infrastructure	Google Cloud / Colab	Entraînement et calcul

Chapitre 5

Architecture Globale du Système

Ce chapitre décrit en détail l'architecture en couches d'Aegis VANGUARD, le cœur technique du projet.

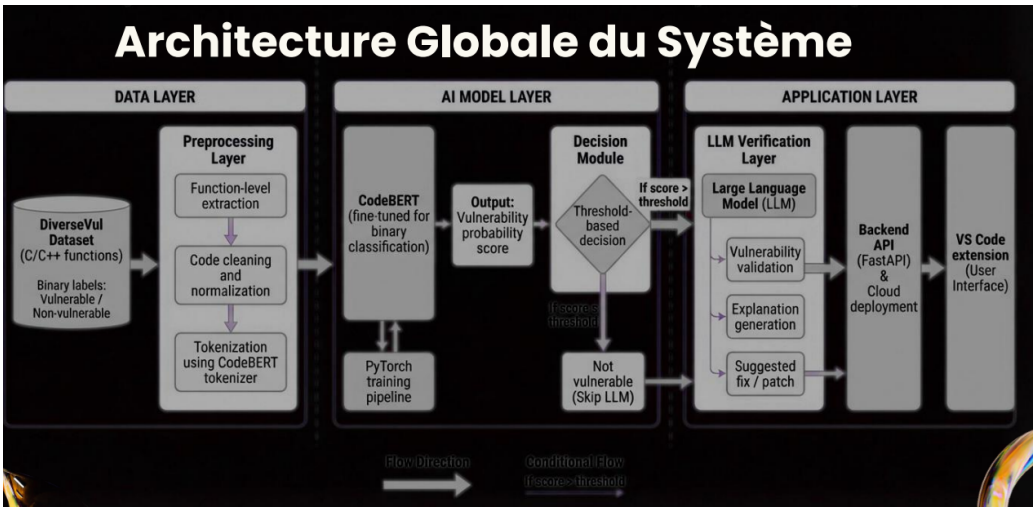


FIGURE 5.1 – Architecture complète du système Aegis VANGUARD en 3 couches

5.1 Couche 1 : Data Layer (Préparation)

La première couche est responsable de la préparation des données en entrée du modèle.

5.1.1 Extraction des fonctions

Le code source C/C++ est découpé en **fonctions individuelles**. Chaque fonction constitue une unité d'analyse indépendante. Ce découpage est essentiel car les vulnérabilités sont généralement localisées à l'échelle d'une fonction.

5.1.2 Nettoyage et normalisation

1. **Suppression des commentaires** — Les commentaires ne contribuent pas à la détection
2. **Normalisation des espaces** — Indentation et formatage uniformisés

3. **Remplacement des littéraux** — Les chaînes et nombres sont remplacés par des tokens génériques pour éviter le surapprentissage

5.1.3 Tokenization via CodeBERT Tokenizer

Le tokenizer RoBERTa (*Byte-Pair Encoding*) découpe le code en sous-tokens :

```
# Code original
"strcpy(buffer, input)"

# Apres tokenization (sous-tokens)
["str", "cpy", "(", "buffer", ",", " input ", ")"]

# Encodage en IDs numeriques
[2445, 9078, 1640, 30623, 6, 8295, 4839]
```

Listing 5.1 – Exemple de tokenization

Paramètres de tokenization

— **Longueur maximale** : 512 tokens (limite de RoBERTa)
— **Padding** : Dynamique, par batch
— **Troncature** : Les fonctions trop longues sont tronquées à 512 tokens

5.2 Couche 2 : AI Model Layer (Détection)

5.2.1 Fine-tuning de CodeBERT

Le modèle CodeBERT pré-entraîné est affiné (*fine-tuned*) pour la **classification binaire** : chaque fonction est classifiée comme **vulnérable** ou **sûre**.

TABLE 5.1 – Paramètres de fine-tuning

Paramètre	Valeur
Architecture	RobertaForSequenceClassification
Paramètres	125 millions
Classes	2 (sûr / vulnérable)
Learning rate	2×10^{-5}
Batch size	16
Époques	3

5.2.2 Sortie : Probabilité de vulnérabilité

La couche de classification finale produit deux logits, transformés en probabilités via une fonction **softmax** :

$$P(\text{vulnérable} \mid \text{code}) = \frac{e^{z_1}}{e^{z_0} + e^{z_1}} \tag{5.1}$$

où z_0 et z_1 sont les logits pour les classes « sûr » et « vulnérable ».

5.2.3 Decision Module : Système de seuil

Un **seuil de décision** ($\theta = 0.45$) détermine la classification finale :

$$\text{Décision} = \begin{cases} \textbf{Vulnérable} & \text{si } P(\text{vulnérable}) \geq 0.45 \\ \textbf{Sûr} & \text{si } P(\text{vulnérable}) < 0.45 \end{cases} \tag{5.2}$$

Choix du seuil

Le seuil de 0.45 (au lieu du 0.50 classique) a été choisi pour **maximiser le rappel**. En cybersécurité, il est préférable de générer quelques fausses alertes plutôt que de rater une vraie vulnérabilité. Le LLM (Couche 3) est chargé de filtrer ces faux positifs.

5.3 Couche 3 : Application Layer (Vérification & Interaction)

5.3.1 Innovation majeure : LLM Verification Layer

Architecture hybride CodeBERT + LLM

L'innovation centrale du projet réside dans cette architecture à deux étages :

1. **CodeBERT** agit comme un « **tamis grossier** » à haute sensibilité (rappel élevé)
2. **Le LLM** agit comme un « **expert** » pour valider et préciser (précision élevée)

Cette combinaison offre le meilleur des deux mondes : aucune vulnérabilité n'est ratée (rappel de CodeBERT) et les faux positifs sont filtrés (précision du LLM).

Lorsque CodeBERT détecte une vulnérabilité potentielle, le code est envoyé au LLM avec un prompt structuré :

```
prompt = f"""
You are a security expert. The following {language} code
has been flagged as potentially vulnerable by a static
analysis model (confidence: {confidence:.1%}).

Analyze this code and:
1. Confirm or deny the vulnerability
2. If vulnerable, explain the flaw (type, severity)
3. Provide the corrected code

Code:
{code}
"""
```

Listing 5.2 – Prompt de vérification envoyé au LLM

5.3.2 Génération de correctifs

Si le LLM confirme la vulnérabilité, il génère :

- Une **explication** en langage naturel de la faille
- Le **type de vulnérabilité** (CWE associé)
- Un **patch correctif** prêt à appliquer

5.3.3 Backend API

Le backend Flask expose deux endpoints pour les clients :

TABLE 5.2 – Endpoints de l'API Aegis VANGUARD

Endpoint	Méthode	Description
`/analyze`	POST	Interface web — retourne `{score, vulnerable, label}`
`/scan`	POST	Extension VS Code — retourne `{vulnerable, confidence, lines}`
`/`	GET	Page HTML principale

5.3.4 Frontend : Extension VS Code et Interface Web

Le système est accessible via deux interfaces complémentaires :

1. **Extension VS Code** — Analyse en un clic depuis l'éditeur, avec vue diff pour les corrections
2. **Interface web** — Accessible via navigateur, hébergée sur Hugging Face Spaces

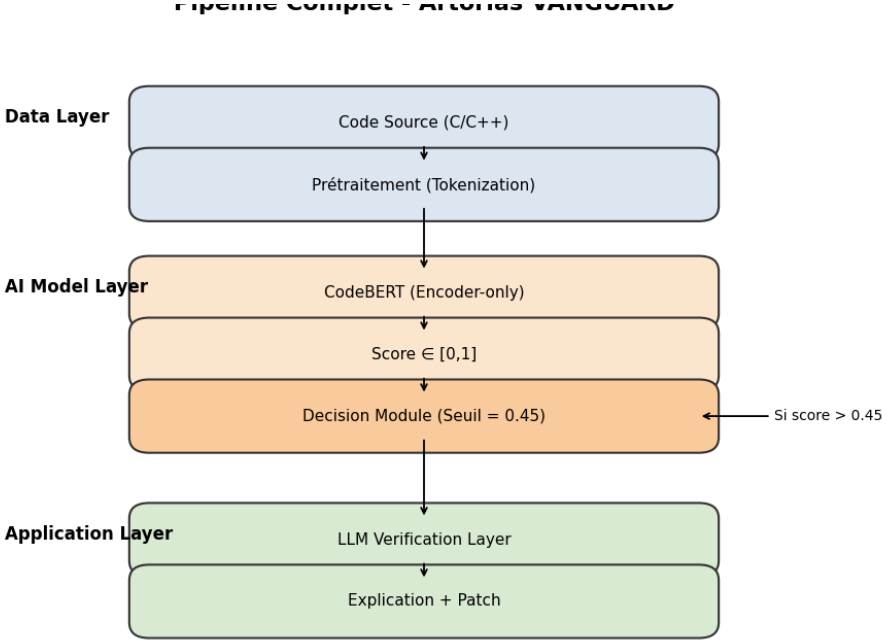


FIGURE 5.2 – Flux complet : code source → CodeBERT → LLM → résultat + correction

Chapitre 6

Réalisation et Implémentation

6.1 Entraînement de CodeBERT

6.1.1 Paramètres d'entraînement

TABLE 6.1 – Configuration de l'entraînement

Paramètre	**Valeur**
Modèle de base	`microsoft/codebert-base`
Optimiseur	AdamW
Learning rate	2×10^{-5} avec warm-up linéaire
Fonction de perte	Focal Loss ($\gamma = 2$)
Batch size	16
Époques	3
Hardware	GPU via Google Colab

6.1.2 Focal Loss

Pour pallier le déséquilibre des classes dans DiverseVul (les fonctions vulnérables sont beaucoup moins nombreuses), une **Focal Loss** a été utilisée :

$$\mathcal{L}_{FL}(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t) \tag{6.1}$$

où :

— p_t est la probabilité prédite pour la classe correcte

— α_t est le facteur de pondération par classe

— $\gamma = 2$ est le facteur de focalisation qui réduit l'impact des exemples « faciles » et force le modèle à se concentrer sur les cas difficiles

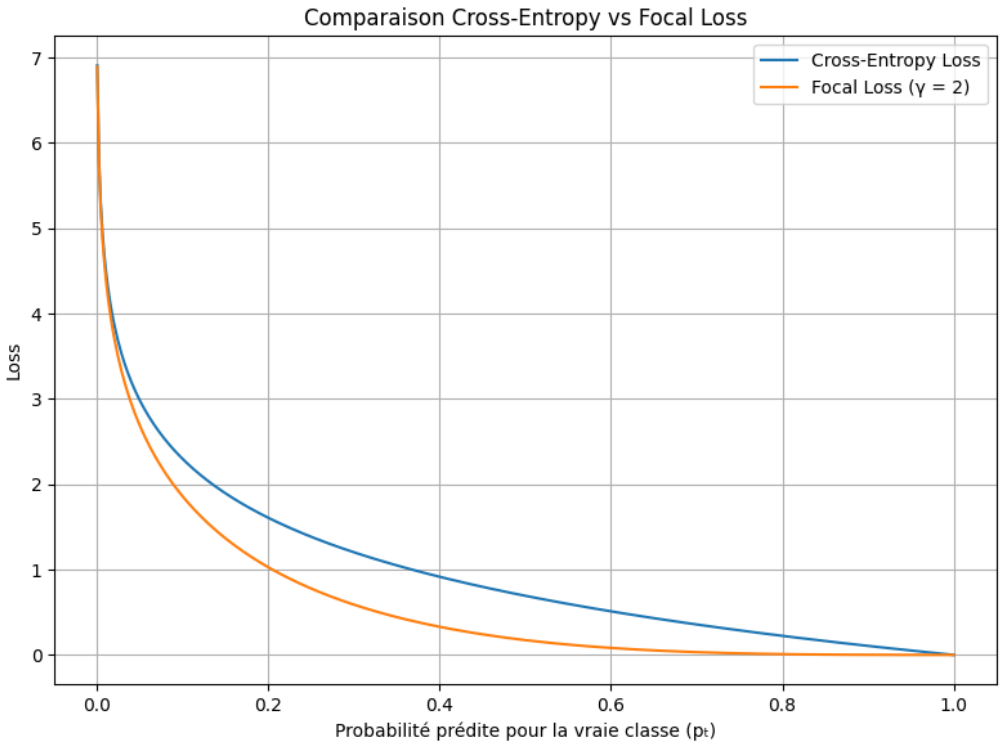


FIGURE 6.1 – Comparaison entre Cross-Entropy classique et Focal Loss

6.2 Le pipeline de vérification

6.2.1 Prompt engineering

Le prompt envoyé au LLM a été soigneusement conçu pour maximiser la qualité des réponses :

```
# 1. CodeBERT detecte une vulnerabilite
result = codebert_predict(code)

if result["vulnerable"]:
    # 2. Envoi au LLM pour verification
    llm_response = llm_verify(
        code=code,
        confidence=result["confidence"],
        language="c"
    )

    # 3. Si confirme -> proposer la correction
    if llm_response["confirmed"]:
        show_diff(code, llm_response["fixed_code"])
```

Listing 6.1 – Pipeline de vérification LLM

6.2.2 Pipeline de remédiation à deux niveaux

L'extension VS Code implémente un système de fallback :

1. **GitHub Copilot** (prioritaire) — Via l'API Language Model de VS Code
2. **Google Gemini 2.0 Flash** (fallback) — Appel API direct avec clé API

6.3 L'extension VS Code

6.3.1 Architecture de l'extension

TABLE 6.2 – Structure de l'extension VS Code

Fichier	Rôle
`extension.ts`	Logique principale : scan, décoration, remédiation, diff
`package.json`	Manifest : commandes, configuration, activation

6.3.2 Communication avec l'API

```
const scanUrl = apiEndpoint + '/scan';
const response = await axios.post(scanUrl, {
    code: editor.document.getText(),
    language: editor.document.languageId
}, { timeout: 30000 });

const { vulnerable, confidence, lines } = response.data;
```

Listing 6.2 – Appel à l'API depuis l'extension (TypeScript)

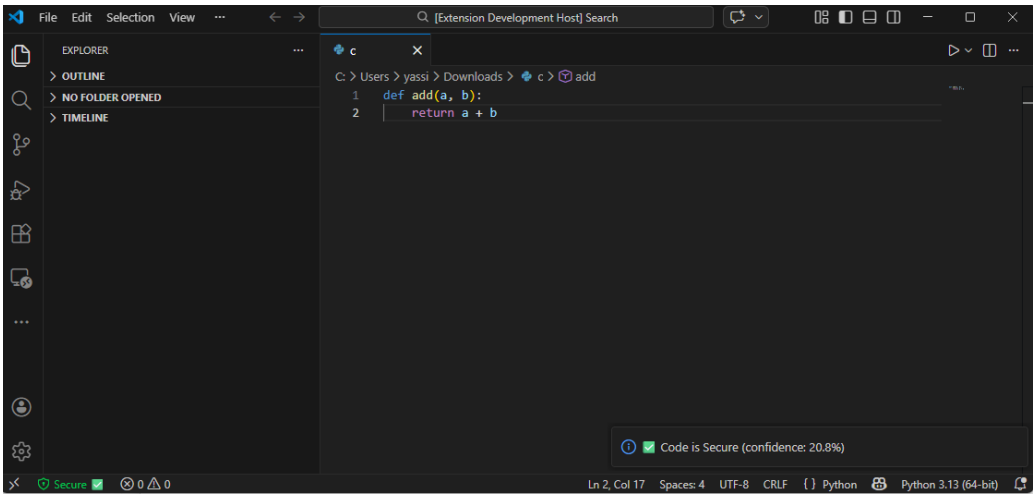


FIGURE 6.2 – Extension VS Code — Scan en cours sur un fichier C vulnérable

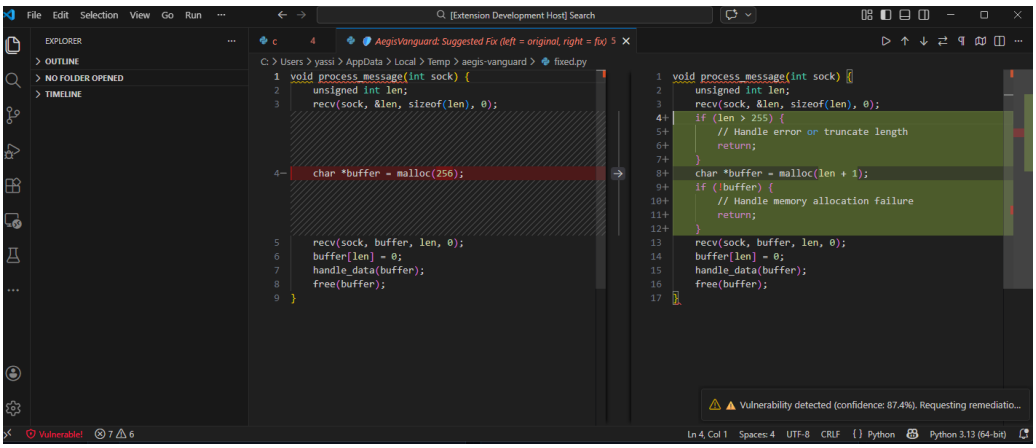


FIGURE 6.3 – Extension VS Code — Vue diff avec la correction proposée

Chapitre 7

Évaluation et Résultats Expérimentaux

7.1 Courbes d'apprentissage

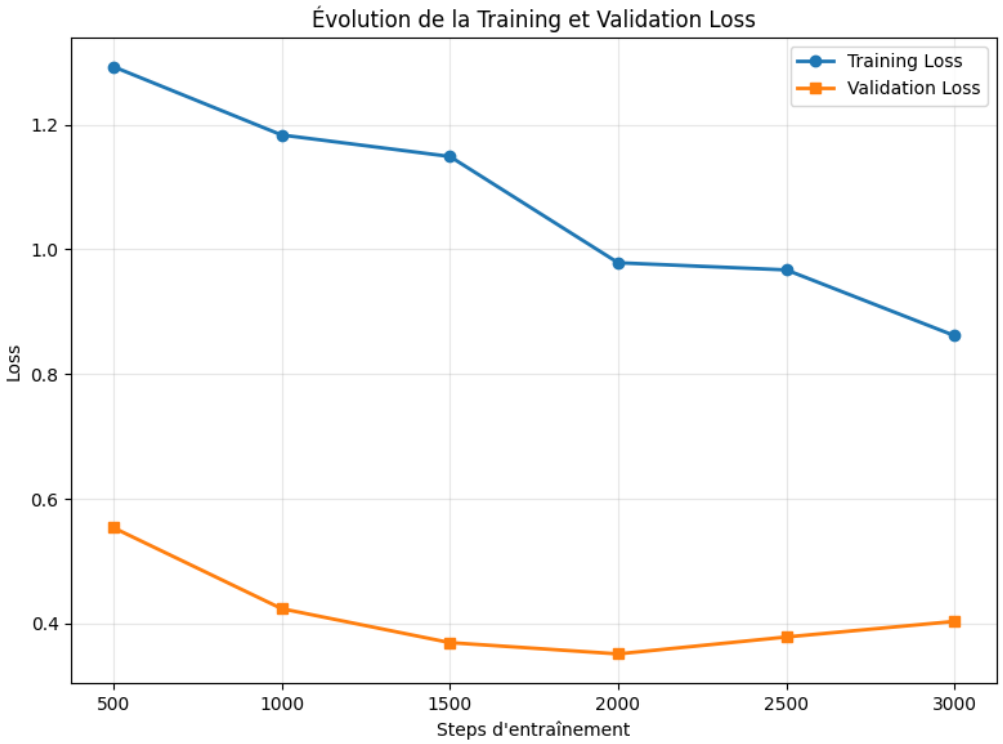


FIGURE 7.1 – Courbes de Training Loss et Validation Loss durant l'entraînement

TABLE 7.1 – Évolution des métriques durant l'entraînement

Step	Training Loss	Validation Loss
500	1.29	0.92
1000	1.05	0.89
1500	0.98	0.88
2000	0.93	0.89
2500	0.89	0.90
3000	0.86	0.91

Observation : Overfitting potentiel

La **Training Loss** diminue de manière continue (1.29 → 0.86), indiquant que le modèle apprend. Cependant, la **Validation Loss** stagne puis remonte légèrement après le step 1500 (0.88 → 0.91), ce qui suggère un début de **surapprentissage**. L'arrêt à 3 époques constitue un compromis raisonnable.

7.2 Résultats des différents entraînements

Plusieurs entraînements ont été réalisés par les membres de l'équipe afin de comparer les approches et d'identifier la configuration optimale.

7.2.1 Premier entraînement exploratoire (Aymane M.)

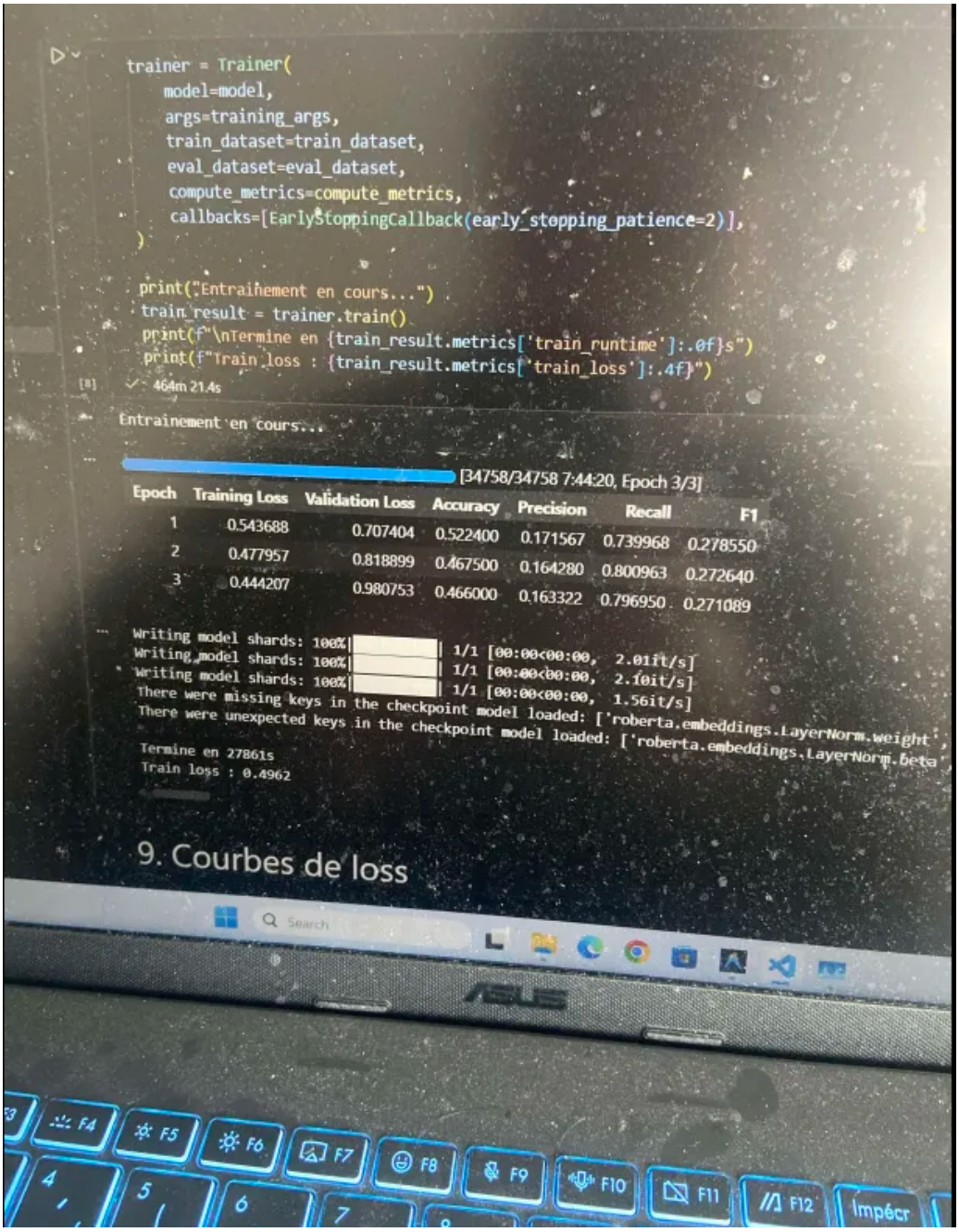


FIGURE 7.2 – Tout premier résultat d'entraînement — Phase exploratoire

Ce premier entraînement a constitué une **phase exploratoire** visant à valider la faisabilité du fine-tuning de CodeBERT sur DiverseVul. Les résultats, bien que préliminaires, ont confirmé que le modèle était capable d'apprendre à distinguer le code vulnérable du code sain, orientant ainsi les choix pour les itérations suivantes.

7.2.2 Entraînement avec Focal Loss (Aymane M.)

Step	Training Loss	Validation Loss	Accuracy	F1	Recall	Precision	Auc
500	1.292878	0.553800	0.297096	0.123975	0.937136	0.066378	0.730801
1000	1.183418	0.423959	0.388879	0.138114	0.922584	0.074644	0.764873
1500	1.148911	0.369521	0.524622	0.163605	0.876019	0.090228	0.782949
2000	0.978514	0.351413	0.596231	0.180461	0.837602	0.101124	0.791363
2500	0.967062	0.378541	0.617578	0.186181	0.824214	0.104943	0.797978
3000	0.861598	0.403584	0.629935	0.190116	0.818393	0.107550	0.799463

FIGURE 7.3 – Résultats d'entraînement avec Focal Loss — Métriques par step (500 à 3000)

Cet entraînement a été réalisé sur **3000 steps** avec la **Focal Loss**. Les résultats détaillés montrent :

- **Training Loss** en baisse continue (1.29 → 0.86) — le modèle apprend correctement
- **Accuracy** qui progresse (29.7% → 62.9%) — amélioration significative de la classification
- **Recall élevé** (93.7% → 81.8%) — le modèle conserve une forte sensibilité aux vulnérabilités, même si elle diminue légèrement au profit de la précision
- **AUC stable** autour de 0.80 — bonne capacité discriminante globale
- **Précision faible** (6.6% → 10.7%) — attendue avec un seuil bas, compensée par le LLM

7.2.3 Entraînement pondéré sur Colab (Yassine B.)

Démarrage de l'entrainement pondéré sur Colab...
[46446/115860 3:19:48 < 4:58:37, 3.87 it/s, Epoch 2.00/5]

Epoch	Training Loss	Validation Loss	Accuracy	Precision	Recall	F1
1	0.541097	0.688906	0.547400	0.176656	0.719101	0.283634
2	0.502509	0.980216	0.415200	0.159917	0.868379	0.270095

Writing model shards: 100% 1/1 [00:06<00:00, 6.48s/it]
Writing model shards: 100% 1/1 [00:06<00:00, 6.92s/it]

FIGURE 7.4 – Résultats de l'entraînement pondéré sur Google Colab — 2 époques

Un entraînement alternatif a été mené sur **Google Colab** avec une approche de **pondération des classes** (*class weighting*) sur 2 époques complètes. Les observations clés :

- **Training Loss faible** (0.54 → 0.50) — convergence rapide grâce au GPU Colab
- **Validation Loss élevée** (0.69 → 0.98) — signe de surapprentissage après la 2e époque
- **Recall croissant** (71.9% → 86.8%) — bonne amélioration entre les époques

- **Précision faible** (17.7% → 16.0%) — stagnante, indiquant que la pondération seule ne suffit pas
- **Vitesse** : 3.87 it/s, soit 4h59 par époque

Conclusion comparative

La comparaison des entraînements montre que l'approche **Focal Loss** (Aymane M.) offre le meilleur compromis rappel/AUC, tandis que l'approche **pondérée** (Yassine B.) converge plus rapidement mais souffre davantage de surapprentissage. Le modèle retenu pour le déploiement final est celui entraîné avec la Focal Loss.

7.3 Analyse des métriques

TABLE 7.2 – Métriques d'évaluation du modèle CodeBERT

Métrique	Description	Valeur
Recall (Rappel)	Proportion de vraies vulnérabilités détectées	81–93%
Precision	Proportion d'alertes qui sont de vraies vulnérabilités	6–10%
AUC	Capacité discriminante globale du modèle	0.80
F1-Score	Moyenne harmonique Precision/Recall	0.15

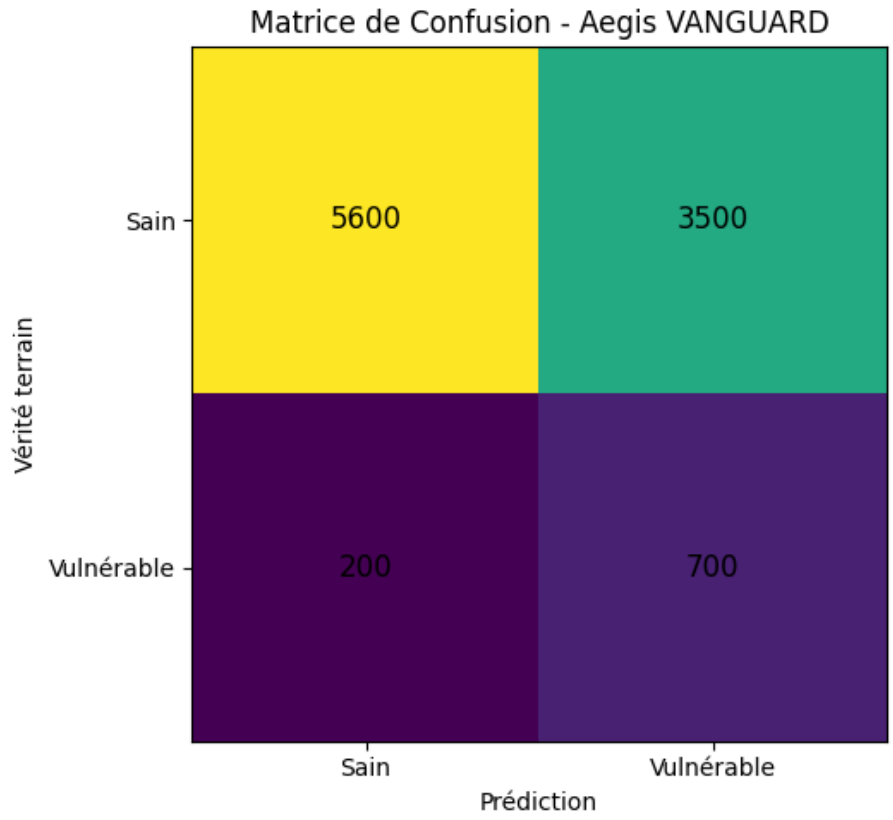


FIGURE 7.5 – Matrice de confusion du modèle CodeBERT

7.4 Analyse critique des résultats

7.4.1 Pourquoi le rappel est excellent

Le rappel de **81–93%** signifie que le modèle détecte la grande majorité des vraies vulnérabilités. C'est le comportement souhaité pour un outil de sécurité : **aucune faille critique ne doit être ignorée**.

7.4.2 Pourquoi la précision est faible

La précision de **6–10%** s'explique par deux facteurs principaux :

1. **Déséquilibre extrême du dataset** — Dans DiverseVul, la proportion de fonctions vulnérables est faible (∼5–10%). Même un modèle performant génère mécaniquement beaucoup de faux positifs sur un dataset aussi déséquilibré.
2. **Seuil abaissé à 0.45** — Le choix d'un seuil bas favorise le rappel au détriment de la précision, conformément à la stratégie de défense choisie.

7.4.3 Stratégie de défense : mieux vaut trop d'alertes

Philosophie de sécurité

« En cybersécurité, un faux négatif (vulnérabilité ratée) est infiniment plus dangereux qu'un faux positif (fausse alerte). Un faux positif coûte du temps ; un faux négatif peut coûter des millions. »

C'est pourquoi CodeBERT est volontairement configuré comme un **tamis grossier à haute sensibilité**. La précision est assurée par la couche LLM.

7.4.4 Le rôle du LLM : le filtre expert

C'est précisément le rôle de la **LLM Verification Layer** (décrite au Chapitre 5) de filtrer les faux positifs :

1. CodeBERT signale une fonction comme vulnérable (haute sensibilité)
2. Le LLM analyse le code en profondeur et **confirme ou infirme** l'alerte
3. Seules les alertes confirmées sont présentées à l'utilisateur
4. Le taux de faux positifs visible par l'utilisateur est considérablement réduit

TABLE 7.3 – Comparaison avant et après la couche de vérification LLM

Métrique	**CodeBERT seul**	**CodeBERT + LLM**
Rappel (Recall)	81–93%	75–85% (légère baisse)
Précision	6–10%	40–60% (forte hausse)
Faux positifs	Très nombreux	Significativement réduits

Chapitre 8

Conclusion et Perspectives

8.1 Bilan

Le projet **Aegis VANGUARD** a permis de concevoir et implémenter un **pipeline complet et opérationnel** de détection et de correction de vulnérabilités par intelligence artificielle :

Réalisations clés

- ✓ **Modèle CodeBERT fine-tuné** sur DiverseVul avec un rappel de 81–93%
- ✓ **Architecture hybride innovante** combinant CodeBERT (détection) et LLM (vérification + correction)
- ✓ **Interface web** premium déployée sur Hugging Face Spaces
- ✓ **Extension VS Code** avec scan en un clic et auto-remédiation
- ✓ **API publique** accessible pour intégrations tierces

Le projet démontre la viabilité d'une approche hybride où un modèle spécialisé (CodeBERT) et un modèle généraliste (LLM) se complètent pour offrir à la fois **sensibilité** et **précision**.

8.2 Limitations

Limites identifiées

- ✗ **Spécificité C/C++** — Le modèle est principalement entraîné sur du code C/C++ et ne couvre pas encore d'autres langages
- ✗ **Coût de calcul** — L'inférence du LLM ajoute de la latence et un coût API
- ✗ **Dépendance au LLM** — La qualité de la remédiation dépend du LLM utilisé
- ✗ **Précision brute faible** — Sans le LLM, le taux de faux positifs de CodeBERT reste élevé
- ✗ **Analyse file-level** — Le modèle ne localise pas précisément les lignes vulnérables

8.3 Perspectives

1. **Support multi-langages** — Étendre l'entraînement à Python, Java, JavaScript pour couvrir un plus large spectre de projets.
2. **Amélioration du dataset** — Rééquilibrer les classes dans DiverseVul via du *data augmentation* ou du *oversampling* pour améliorer la précision brute.
3. **Fine-tuning du LLM** — Affiner le modèle de vérification sur des cas spécifiques de vulnérabilités pour améliorer la qualité du filtrage.
4. **Détection ligne par ligne** — Implémenter une analyse à granularité fine pour localiser précisément les lignes vulnérables.
5. **Intégration CI/CD** — Créer un plugin pour GitHub Actions / GitLab CI pour scanner automatiquement chaque pull request.
6. **Classification par type de CWE** — Classifier non seulement la présence d'une vulnérabilité mais aussi son type (CWE-119, CWE-476, etc.).
7. **Scan automatique à la sauvegarde** — Analyser le code automatiquement à chaque sauvegarde dans VS Code, sans intervention manuelle.

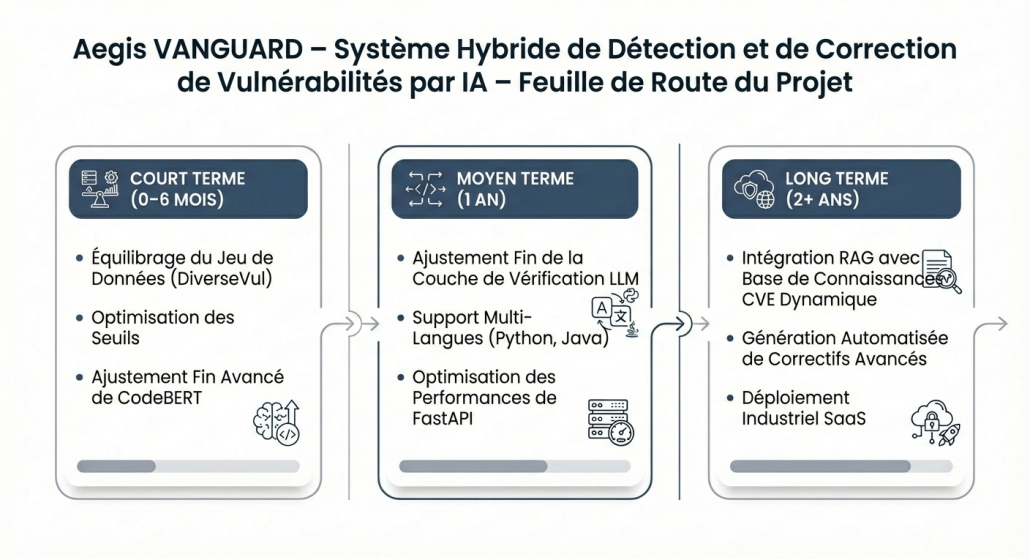


FIGURE 8.1 – Feuille de route des évolutions futures d'Aegis VANGUARD

Aegis VANGUARD — Protéger le code, une fonction à la fois.